

Dissertation on

**"Agentic Discovery of Novel, Data-Driven Composite
Preprocessing of fMRI Data within fMRIPrep Pipelines"**

Submitted in partial fulfilment of the requirements for the award of degree of

**Bachelor of Technology
in
Computer Science & Engineering**

UE23CS320B – Capstone Project Phase - 2

Project ID: PW26_CN_07

Submitted by:

N Riddhi Samitha	PES1UG23AM175
Mohammad Moosa Kalimulla	PES1UG23CS363
Manya Udaya Shetty	PES1UG23CS915
Rengashyam Rajagopalan	PES1UG23EC241

Under the guidance of

Dr. Chaitra N Assistant Professor, Department of CSE
--

January – May 2026

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
FACULTY OF ENGINEERING
PES UNIVERSITY**

(Established under Karnataka Act No. 16 of 2013)
100ft Ring Road, Bengaluru – 560 085, Karnataka, India

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

100 Feet Ring Road, Bengaluru – 560 085, Karnataka, India

FACULTY OF ENGINEERING

CERTIFICATE

This is to certify that the dissertation entitled

"Agentic Discovery of Novel, Data-Driven Composite Preprocessing of fMRI Data within fMRIPrep Pipelines"

is a bonafide work carried out by

N Riddhi Samitha	PES1UG23AM175
Mohammad Moosa Kalimulla	PES1UG23CS363
Manya Udaya Shetty	PES1UG23CS915
Rengashyam Rajagopalan	PES1UG23EC241

in partial fulfilment for the completion of sixth semester Capstone Project Phase - 2 (UE23CS320B) in the Program of Study - Bachelor of Technology in Computer Science and Engineering under rules and regulations of PES University, Bengaluru during the period Jan 2026 – May 2026. It is certified that all corrections/suggestions indicated for internal assessment have been incorporated in the report. The dissertation has been approved as it satisfies the 6th semester academic requirements in respect of project work.

Signature Dr. Chaitra N Guide	Signature Dr. Shylaja S S Chairperson	Signature Dr. K S Sridhar Registrar
--	--	--

DECLARATION

We hereby declare that the Capstone Project Phase - 2 entitled "Agentic Discovery of Novel, Data-Driven Composite Preprocessing of fMRI Data within fMRIPrep Pipelines" has been carried out by us under the guidance of Dr. Chaitra N, Assistant Professor, Department of Computer Science and Engineering, and submitted in partial fulfilment of the course requirements for the award of degree of Bachelor of Technology in Computer Science and Engineering of PES University, Bengaluru during the academic semester Jan - May 2026. The matter embodied in this report has not been submitted to any other university or institution for the award of any degree.

PES1UG23AM175	N Riddhi Samitha	_____
PES1UG23CS363	Mohammad Moosa Kalimulla	_____
PES1UG23CS915	Manya Udaya Shetty	_____
PES1UG23EC241	Rengashyam Rajagopalan	_____

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to Dr. Chaitra N, Assistant Professor, Department of Computer Science and Engineering, PES University, for her continuous guidance, expertise in neuroimaging, and unwavering encouragement throughout the development of this UE23CS320B - Capstone Project Phase – 2. Her insights into fMRI preprocessing and agentic AI were invaluable to the success of this project.

We are grateful to the project coordinator, Prof. Gayathri R S, for organizing, managing, and helping with the entire process.

We take this opportunity to thank Dr. Shylaja S S, Chairperson, Department of Computer Science and Engineering, PES University, for all the knowledge and support we have received from the department. We would like to thank Dr. K.S. Sridhar, Registrar, PES University for his help.

We are deeply grateful to Prof. Jawahar Doreswamy, Chancellor – PES University, Dr. Suryaprasad J, Vice-Chancellor, PES University for providing various opportunities and enlightenment every step of the way. Finally, this project could not have been completed without the continual support and encouragement we have received from our families and friends.

ABSTRACT

Functional Magnetic Resonance Imaging (fMRI) has become a cornerstone of modern neuroscience, enabling researchers to observe brain activity with high spatial resolution. However, the increasing reliance on data-intensive 3D and 4D fMRI scans has exposed a critical limitation in current preprocessing methodologies: the absence of a dynamic, adaptive data processing pipeline. Standard tools such as fMRIPrep apply a static, one-size-fits-all sequence of operations that fails to account for subject-specific variability in anatomy, physiology, and noise characteristics, necessitating frequent manual intervention for quality control and parameter adjustment. This human-in-the-loop requirement introduces inconsistencies and severely compromises the reproducibility of neuroimaging results.

This report presents an Agentic AI framework that transforms fMRI preprocessing from a fixed automation sequence into an intelligent, autonomous workflow. The system employs a Multi-Agent System (MAS) architecture — comprising an Orchestrator, BIDS Agent, Resource Agent, Pipeline Agent, Diagnostic Agent, and Vision Agent — orchestrated using LangGraph. Agents evaluate unique noise patterns, motion characteristics, and signal quality metrics in real-time, dynamically adjusting pipeline parameters and autonomously recovering from runtime errors. The system successfully demonstrated end-to-end autonomous preprocessing of a BIDS-formatted dataset (OpenNeuro ds000102), achieving a quality score of 0.94/1.0 and completing with zero manual interventions after recovering from three critical runtime failures, validating the core agentic design.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1	INTRODUCTION	
2	LITERATURE SURVEY	
3	ARCHITECTURE WITH EXPLANATION	
4	METHODOLOGY	
5	IMPLEMENTATION (Salient Modules)	
6	RESULTS	
7	CONCLUSION AND FUTURE WORK	
8	REFERENCES/BIBLIOGRAPHY	

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 3.1	Multi-Agent System Architecture Flowchart	
Figure 3.2	External Interfaces and System Components	
Figure 3.3	Master Class Diagram	
Figure 3.4	Use Case Diagram	
Figure 6.1	Execution History and Autonomous Recovery Events	
Figure 6.2	System Report Output – Quality Score 0.94/1.0	

LIST OF TABLES

Table No.	Title	Page No.
Table 2.1	Summary of Related Works in Automated fMRI Preprocessing	
Table 5.1	Technologies and Tools Used	
Table 5.2	Dataset Summary	

CHAPTER 1

INTRODUCTION

1.1 Background

Functional Magnetic Resonance Imaging (fMRI) is a non-invasive neuroimaging technique that measures brain activity by detecting changes in blood oxygenation levels, a phenomenon known as the Blood Oxygenation Level-Dependent (BOLD) response. By capturing 4D time-series volumes of brain activity, fMRI has become a cornerstone of modern cognitive neuroscience, enabling researchers to map neural correlates of perception, cognition, emotion, and disease.

Raw fMRI data, however, contains a mixture of neural signal and confounding artifacts arising from head motion, physiological noise (cardiac and respiratory cycles), scanner instabilities, and thermal noise. Before any scientific analysis can be conducted, raw data must pass through a multi-step preprocessing pipeline that corrects these confounds and brings data into a standardized anatomical space. The quality of preprocessing directly determines the quality of all downstream scientific findings.

1.2 Problem Statement

The current standard for fMRI preprocessing, fMRIPrep, provides a reproducible and modular pipeline. However, its design is fundamentally static: the same sequence of operations and parameter values are applied to every subject, regardless of that subject's unique data characteristics. This creates a critical bottleneck in large-scale neuroimaging studies:

- Static pipelines cannot adapt to subjects with unusual anatomy, atypical motion profiles, or physiological noise patterns that differ from the population average.
- Manual quality control (QC) is required after every run, creating a human bottleneck that is slow, subjective, and introduces operator-dependent variability.
- Pipeline failures due to missing metadata, memory constraints, or file system errors require expert diagnosis and manual restart, often hours or days later.
- Parameter choices such as the number of noise components to regress are made once and applied universally, potentially over-fitting or under-fitting for specific subjects.

1.3 Proposed Solution

This project addresses these limitations by developing an Agentic AI framework for autonomous fMRI preprocessing. The system employs a Multi-Agent System (MAS) architecture where specialized AI agents collaboratively manage every phase of the preprocessing lifecycle — from initial data validation through quality assurance. Rather than static parameter sets, agents evaluate scan-specific data characteristics in real-time and make informed, logged decisions.

The key innovation is the transformation of preprocessing from a passive automation script into an active, goal-directed workflow. When the pipeline encounters errors, the Diagnostic Agent performs root-cause analysis, and the Recovery Agent autonomously applies targeted fixes and restarts the pipeline — without human intervention.

1.4 Scope and Objectives

The specific objectives of this Phase 2 implementation are:

- Implement a working multi-agent orchestration system using LangGraph for stateful, crash-aware workflow management.
- Develop specialized agents for BIDS validation, resource monitoring, fMRIPrep command generation, diagnostic analysis, and visual quality assurance.
- Demonstrate autonomous end-to-end preprocessing on a real BIDS dataset with zero manual intervention.
- Establish a foundation for Reinforcement Learning-based pipeline optimization in future phases.

CHAPTER 2

LITERATURE SURVEY

2.1 Standard fMRI Preprocessing Pipelines

Esteban et al. (2019) introduced fMRIPrep as a robust, analysis-agnostic preprocessing workflow for fMRI data [1]. The tool wraps established neuroimaging tools including ANTs, FSL, and FreeSurfer into a modular, reproducible pipeline using the Nipype workflow framework. Its key contributions are standardized outputs in the BIDS Derivatives format and a configurable, modular architecture. However, fMRIPrep's core design remains static: parameter choices are fixed at invocation time and applied uniformly across all subjects. This "one-size-fits-all" approach is its primary limitation when applied to heterogeneous, large-scale datasets.

Luppi et al. (2024) conducted a systematic evaluation of fMRI preprocessing pipelines for functional connectomics, demonstrating that pipeline choice introduces substantial variability in downstream connectivity estimates [3]. Their findings underscore the need for principled, data-driven selection of preprocessing strategies rather than arbitrary application of a single fixed pipeline.

2.2 Adaptive and Automated Preprocessing

Churchill et al. (2015) proposed an automated framework for optimizing preprocessing pipelines in task-based fMRI [7]. Their system evaluated multiple pipeline variants and selected the optimal configuration based on a reproducibility-weighted signal-to-noise metric. While pioneering, their approach relied on exhaustive grid search over predefined configurations rather than intelligent, real-time adaptation. Our work extends this idea by introducing an agent that actively diagnoses failure modes and applies targeted corrective actions.

Ren et al. (2025) presented DeepPrep, an accelerated preprocessing pipeline powered by deep learning for anatomical segmentation [11]. DeepPrep demonstrates that deep learning can replace traditional atlas-based segmentation with significant speed improvements. Our project complements this direction by focusing on the orchestration and decision-making layer above the processing engine, rather than replacing the engine itself.

2.3 Noise Correction and Quality Control

Pruim et al. (2015) developed ICA-AROMA, a data-driven method for removing motion artifacts from fMRI data using Independent Component Analysis [26]. ICA-AROMA automatically classifies components as signal or noise based on spatial and temporal features. This work inspired our Intelligent Noise Filtering design goal, where the agent selects appropriate denoising strategies rather than applying a fixed regressor set.

Wang et al. (2024) provided a continuous evaluation of denoising strategies in resting-state fMRI connectivity using fMRIPrep and Nilearn, demonstrating that CompCor component selection significantly impacts functional connectivity estimates [5]. Their findings directly motivate our design of a QualityAssessor agent that monitors Signal-to-Noise Ratio (SNR) as a reward proxy.

2.4 Reinforcement Learning in Medical Imaging

Pineda et al. (2020) applied Reinforcement Learning to active MRI k-space sampling, framing the acquisition problem as a sequential decision process [9]. The RL agent learned policies that outperformed handcrafted sampling strategies. This work provides a strong theoretical basis for our application of RL to the preprocessing decision problem, where the agent must sequentially select preprocessing operations to maximize data quality.

Yang (2025) demonstrated that deep neural networks can perform adaptive spatial smoothing of task-fMRI data, with smoothing parameters adjusted based on local signal characteristics [6]. This represents a component-level application of the adaptive processing philosophy that our project generalizes to the full pipeline level.

2.5 Summary of Related Works

Table 2.1 summarizes the key related works and their relationship to this project.

Reference	Contribution	Limitation	Our Extension
fMRIPrep [1]	Standardized, modular preprocessing pipeline	Fully static; manual QC required	Agentic orchestration layer on top
Churchill et al. [7]	Automated pipeline optimization via grid search	Exhaustive search; no real-time adaptation	Agent-based targeted recovery
ICA-AROMA [26]	Data-driven motion artifact removal	Single-component method; fixed classification	Multi-strategy noise filtering selection
Pineda et al. [9]	RL for MRI k-space sampling	Applied at acquisition, not preprocessing	RL applied to full preprocessing pipeline

DeepPrep [11]	DL-accelerated preprocessing engine	Replaces engine, not orchestration	Intelligent orchestration over the engine
---------------	-------------------------------------	------------------------------------	---

Table 2.1: Summary of Related Works in Automated fMRI Preprocessing

CHAPTER 3

ARCHITECTURE WITH EXPLANATION

3.1 Overall System Design

The Agentic fMRIPrep system is built on a Multi-Agent System (MAS) architecture integrated into a closed-loop preprocessing framework. The system follows the principle of stateful orchestration: a central Orchestrator maintains global pipeline state and dispatches control to specialized agents based on the outcome of each processing step. The architecture is implemented using LangGraph, a library for building stateful, multi-actor applications with language models.

The fundamental design insight is to separate concerns: the fMRIPrep engine handles the computation-intensive neuroimaging operations, while the agent layer handles all decision-making, quality assessment, diagnosis, and recovery. This separation ensures backward compatibility — the agentic components can be added to or removed from an existing fMRIPrep deployment without disrupting the core processing engine.

3.2 LangGraph State Machine

The core of the orchestration is a directed graph defined in LangGraph with the following topology:

```
Planner → Executor → (success) → Vision Agent → END
                        |
                        └─ (fail) → Detective → Engineer → Executor (loop)
```

The shared AgentState TypedDict carries five fields across all graph nodes: the current Docker command string, the fMRIPrep log output, an append-only history list, an events list for provenance, the pipeline status string, an attempt counter, and a recovery_changed flag that prevents infinite retry loops when no new fixes are available. The retry loop is bounded by a configurable max_recovery_attempts parameter (default: 3).

3.3 Agent Descriptions

3.3.1 Orchestrator

The Orchestrator is the master controller. It manages global state, decides when to switch between execution mode and diagnostic/recovery mode, and enforces the retry budget. On each iteration, it

reads the pipeline status and routes control to the appropriate next node. It also handles the final reporting step, aggregating all events and quality metrics into a structured system report.

3.3.2 BIDS Agent

The BIDS Agent is responsible for metadata validation and BIDS file structure compliance. When fMRIPrep rejects a dataset due to missing or malformed metadata, the BIDS Agent detects the specific missing field from the error log, programmatically injects the correct value into the JSON sidecar file, and flags the pipeline for restart. In the Phase 2 run, the BIDS Agent demonstrated self-healing by detecting a missing RepetitionTime (TR) field and injecting the correct value without human intervention.

3.3.3 Resource Agent

The Resource Agent continuously monitors hardware metrics — specifically CPU utilization and RAM consumption — while the Docker container runs. When a memory-related crash (exit code 137, indicating an Out-of-Memory kill signal from the operating system) is detected, the Resource Agent autonomously reconfigures the Docker invocation with the `--low-mem` flag and reduces thread count to 1, allowing the pipeline to continue on resource-constrained hardware.

3.3.4 Pipeline Agent (Config Agent)

The Pipeline Agent generates the correct fMRIPrep Docker command. It first attempts to use a Retrieval-Augmented Generation (RAG) chain built over the official fMRIPrep and BIDS specification PDF manuals. If no LLM or documentation is available, it falls back to a deterministic, rule-based command builder that reads all parameters from the central `config.yaml` file. The agent handles complex CLI argument construction for both fMRIPrep and FreeSurfer, including subject-specific flags such as `--anat-only`, `--sloppy`, `--low-mem`, and fieldmap-related options.

3.3.5 Diagnostic Agent

The Diagnostic Agent reads the fMRIPrep log output upon pipeline failure and performs root-cause analysis. It uses a priority-ordered list of regex heuristics to match known error patterns (memory errors, missing metadata, fieldmap issues, naming conflicts, Docker daemon errors). When an LLM is available via the Groq API, it augments the heuristic diagnosis with a natural language explanation. The agent outputs a structured diagnosis including the identified error category and a list of suggested corrective flags.

3.3.6 Recovery Agent

The Recovery Agent translates the Diagnostic Agent's output into concrete modifications to the Docker command. It maintains a set of fix rules that map diagnosis patterns to command transformations — for example, an OOM diagnosis triggers addition of the `--low-mem` flag, reduction of `--mem_mb`, and restriction of `--nprocs` to 1. The agent performs surgical modifications to the command, preserving all other flags while applying the minimum set of changes required to address the diagnosed failure.

3.3.7 Vision Agent

The Vision Agent runs after a successful fMRIPrep execution. It parses the generated HTML quality report and extracts key QC images, including the skull-stripped T1w image, brain mask overlay, normalization GIF, and nuisance mask figures. Using computer vision analysis, it generates a Pass/Fail quality score with confidence. In the Phase 2 run, the Vision Agent confirmed that skull stripping correctly removed non-brain tissue and that normalization to the MNI152NLin2009cAsym template was properly aligned, producing a final quality score of 0.94/1.0.

3.4 Design Constraints

The architecture is governed by four key design constraints. Determinism and Reproducibility require that pipeline outputs are identical for identical inputs; the learning policies are frozen at inference time to ensure this. Auditability and Provenance require that all agent decisions are logged with explicit metrics and rationale, making the system auditable for clinical use. Analysis-Agnostic Optimization ensures that agent objectives rely only on general data-quality proxies rather than hypothesis-specific metrics, preserving applicability across different downstream analyses. Backward Compatibility requires that agentic components are modular and non-disruptive, allowing deployment as an optional wrapper around existing fMRIPrep installations.

CHAPTER 4

METHODOLOGY

4.1 Design Approach

The project employs a Multi-Agent Systems (MAS) approach within a closed-loop preprocessing framework. The core design philosophy treats fMRI preprocessing as a sequential decision problem rather than a fixed procedure. This framing enables the application of Reinforcement Learning concepts: at each decision point (e.g., choosing motion correction strategy, selecting noise components), the agent selects an action, observes the resulting data quality, and updates its policy accordingly.

In Phase 2, the RL component is implemented as a rule-based policy derived from domain expertise, with infrastructure for online learning established for future phases. The reward function is defined as a composite of Signal-to-Noise Ratio (SNR), normalized cross-correlation with the MNI template, and framewise displacement (FD) metrics extracted from the fMRIPrep outputs.

4.2 Dataset and Data Engineering

Two datasets were used in the project. The primary dataset for tool exploration and QC analysis is the OpenNeuro ds004331 dataset: "The spatiotemporal neural dynamics of object recognition for natural images and line drawings (fMRI)." The implementation and testing dataset is OpenNeuro ds000102: a BIDS-formatted, event-related fMRI dataset collected around 2007–2010, containing structural MRI (T1-weighted) and functional BOLD data with a total size of approximately 1.65 GB.

A critical data engineering contribution in Phase 2 is the "Toy Brain" strategy. Full 4D fMRI datasets are too large to run on consumer hardware in a reasonable time. Using the Python nibabel library, the team implemented a script that crops the full anatomical MRI scan into 64x64x64 voxel cubes. This reduces the computational load by approximately 85% while preserving the mathematical relationships between voxels that the pipeline agents need to operate on. The cropped data maintains valid BIDS format and is processed identically to full-resolution data by fMRIPrep.

4.3 Agent Decision Framework

Each agent follows a three-phase decision cycle: Observe, Reason, Act. In the Observe phase, the agent reads its designated input — whether a log file, hardware metrics, HTML report, or configuration file. In the Reason phase, the agent applies its decision logic, either through heuristic rules or LLM inference. In the Act phase, the agent either modifies the pipeline state, injects metadata, updates the command string, or signals the Orchestrator.

The Orchestrator implements a meta-reasoning layer that monitors the overall trajectory of the pipeline. It detects when consecutive recovery attempts are not making progress (using the `recovery_changed` flag) and terminates the loop to prevent infinite retry cycles, flagging the case for human review.

4.4 Quality Assessment Methodology

Quality assessment uses a two-level approach. At the pipeline level, fMRIPrep generates a comprehensive HTML QC report including anatomical tissue boundary overlays, functional-anatomical coregistration GIFs, CompCor cumulative variance plots, nuisance correlation matrices, framewise displacement time series, and carpet plots. The Vision Agent parses these artifacts programmatically.

At the signal level, the QualityAssessor agent computes a scalar quality score from the extracted QC metrics. The score is a weighted combination of: skull-stripping mask boundary accuracy (assessed by analyzing the overlap between the brain mask and cortical edge estimates), normalization quality (assessed by cross-correlation of the normalized T1w with the MNI template), and motion level (FD summary statistics). In Phase 2, this score was calibrated against manual visual inspection to establish the scoring function weights.

4.5 Design Assumptions

The system operates under four key design assumptions. Partial Observability assumes that preprocessing quality can be adequately inferred from QC artifacts and derived metrics, even without access to ground-truth brain signal. Learnable Decision Structure assumes that expert QC decisions exhibit consistent, pattern-based structure that can be captured by heuristic rules and eventually by learned policies. Generalizable Data Quality assumes that analysis-agnostic indicators of data quality (SNR, FD, registration accuracy) are applicable across different datasets and study designs. Episodic Decision Process treats each subject or run as an independent decision episode, enabling RL training without cross-episode dependencies.

CHAPTER 5

IMPLEMENTATION (SALIENT MODULES)

5.1 System Entry Point and Configuration

The system is launched via `main.py`, which parses command-line arguments (participant ID, BIDS directory, memory limits, pipeline flags) and merges them with the central `config.yaml` configuration file. The `Config` object is passed to all agents, ensuring consistent parameterization. The `config.yaml` specifies paths, subject identifiers, pipeline flags (anat-only, sloppy, low-mem), LLM provider (Groq with `llama-3.3-70b-versatile`), embedding model (`all-MiniLM-L6-v2`), and agent behavior (`max_recovery_attempts`, `vision_enabled`).

5.2 Orchestrator Module (`agents/orchestrator.py`)

The Orchestrator module defines the `LangGraph` state machine. It instantiates all agents, defines the `AgentState` `TypedDict`, and constructs the directed graph of node functions. Key implementation details include: the `_resolve_subject_output_dir` function that locates the subject derivatives directory under either flat or `fmriprep/` sub-directory layouts; the `_persist_qa_results` function that writes QA reports and metrics JSON to disk for provenance; and the `_tokenize_command` function that normalizes Docker command strings for comparison across recovery iterations. The graph is compiled and returned as a callable app object.

5.3 Config Agent with RAG (`agents/config_agent.py`)

The Config Agent implements a two-path command generation strategy. In the primary path, it constructs a `LangChain` retrieval-augmented generation chain over the `fMRIPrep` manual and BIDS specification PDFs stored in `data/docs/`. Documents are chunked, embedded using the `all-MiniLM-L6-v2` sentence transformer, and stored in a `ChromaDB` vector database. On invocation, the agent retrieves the most relevant documentation chunks for the current participant's data characteristics and uses the Groq LLM to generate a tailored Docker command. In the fallback path, the `_rule_based_command` function deterministically assembles the command from `Config` values, handling fieldmap detection and flag ordering.

5.4 Diagnostic Agent (`agents/diagnostic_agent.py`)

The Diagnostic Agent maintains a priority-ordered list of heuristic rules, each defined as a (priority, regex_pattern, human_label, suggested_fixes) tuple. When the pipeline fails, the agent scans the error log against all patterns and selects the highest-priority match. The seven implemented heuristics cover: missing readout timing metadata (priority 100), Out-of-Memory crashes (priority 95), missing RepetitionTime (priority 90), fieldmap errors (priority 80), FreeSurfer license issues (priority 70), file system naming conflicts (priority 60), and Docker daemon errors (priority 50). When a Groq API key is available, the matched heuristic is passed to the LLM for natural language elaboration.

5.5 Recovery Agent (agents/recovery_agent.py)

The Recovery Agent implements fix rules as a list of (regex, lambda) pairs. Each lambda takes the current command token list and the Config object and applies targeted modifications. The `_ensure_flag` and `_replace_flag` helper functions provide safe, idempotent flag manipulation. OOM recovery adds `--low-mem`, sets `--mem_mb` to the configured limit, and reduces `--nprocs` to 1. Missing TR recovery adds `--skip-bids-validation`. Fieldmap recovery adds `--use-syn-sdc` or `--ignore-fieldmaps`. Naming conflict recovery injects FreeSurfer-specific skip flags (`-noglobcareg`, `-nocanorm`). After applying fixes, the agent sets `recovery_changed=True` if at least one modification was made, enabling the Orchestrator to detect non-progress.

5.6 Technologies Used

Technology / Tool	Primary Role	Why Essential
LangGraph	Stateful Multi-Agent Orchestration	Enables stateful graph topology with crash-aware retry loops
fMRIPrep / FreeSurfer	Neuroimaging Processing Engine	Core tool for anatomical cleaning and surface reconstruction
BIDS Standard	Standardized Metadata & File Structure	Allows BIDS Agent to detect and self-heal structural errors
Python nibabel	Data Engineering & Toy Brain Strategy	Crops full MRI scans to 64x64x64 voxels, reducing compute by 85%
Groq + LLaMA 3.3	LLM Reasoning Engine	High-speed inference for diagnosis and RAG-based command generation
ChromaDB	Vector Store for RAG	Persistent embedding store for fMRIPrep and BIDS documentation
Docker	Containerized Pipeline Execution	Reproducible, isolated execution environment for fMRIPrep

Table 5.1: Technologies and Tools Used

5.7 Dataset Summary

Property	Details
Dataset Name	OpenNeuro ds000102
Source	OpenfMRI (now OpenNeuro)
Size	1.65 GB
Type	BIDS-formatted: Structural MRI (T1w) + Functional BOLD (event-related task)
Collection Period	Cross-sectional, estimated 2007–2010
Toy Brain Crop	64x64x64 voxels (85% compute reduction)

Table 5.2: Dataset Summary

CHAPTER 6

RESULTS

6.1 Execution Summary

The Agentic fMRIPrep system was tested on subject sub-01 from the OpenNeuro ds000102 dataset using the toy-brain strategy (64x64x64 voxel crop). The system completed end-to-end preprocessing autonomously, requiring zero manual interventions despite encountering three distinct critical failures during the course of execution. The final system report was generated at timestamp 2026-02-17 02:42:34, with FINAL STATUS: COMPLETED.

6.2 Autonomous Recovery Events

Three critical failure events were encountered and autonomously resolved, demonstrating the system's resilience:

6.2.1 Metadata Failure (Attempt 1)

fMRIPrep rejected the BIDS dataset with an error indicating a missing RepetitionTime (TR) field in the task-specific JSON sidecar. The Diagnostic Agent correctly identified this as a "Missing RepetitionTime in BIDS metadata" event (heuristic priority 90). The Recovery Agent triggered the BIDS Agent, which programmatically injected the correct TR value into the sidecar JSON and added the `--skip-bids-validation` flag to the Docker command. The pipeline was restarted successfully.

6.2.2 Memory Crash (Attempt 2)

During FreeSurfer surface reconstruction (a particularly memory-intensive operation), the MacBook Air development machine ran out of RAM. The Docker container was killed by the operating system with exit code 137 (SIGKILL). The Diagnostic Agent correctly identified this as an "OUT-OF-MEMORY crash" (heuristic priority 95). The Resource Agent reconfigured the Docker invocation with the `--low-mem` flag and `--mem_mb 4000`, and restricted `--nprocs` to 1 to prevent concurrent memory pressure. The pipeline was restarted with these constraints.

6.2.3 FileSystem Naming Conflict (Attempt 3)

A naming conflict in the FreeSurfer output directory caused the final stage of surface reconstruction to crash. The Diagnostic Agent identified this as an "Output filesystem naming conflict" (heuristic priority 60). The Orchestrator autonomously entered recovery mode and the Recovery Agent injected FreeSurfer-specific skip flags (-nogcareg, -nocanorm) to bypass the conflicting registration steps. Critically, the system resumed processing from the 4.6-hour mark rather than restarting from scratch, preserving all intermediate results.

6.3 Quality Assessment Results

Following successful completion, the Vision Agent performed a visual quality audit by comparing the preprocessed T1w output against the raw input. The audit produced the following findings:

- Visual Audit: Preprocessing successful.
- Skull-stripping: Non-brain tissue (neck and dura) correctly REMOVED.
- Normalization: Brain correctly ALIGNED to the MNI152NLin2009cAsym standard template.
- Quality Score: 0.94 / 1.0

The quality score of 0.94/1.0 indicates high-quality preprocessing output. The 0.06 penalty was attributed to minor residual intensity inhomogeneity artifacts in the boundary voxels of the 64x64x64 toy crop, which is expected given the aggressive cropping applied to reduce computational load.

6.4 Output Files

The following key output files were produced in outputs/fmriprep/sub-01/anat/:

- sub-01_desc-preproc_T1w.nii.gz — Preprocessed T1-weighted anatomical image in MNI space
- sub-01_desc-brain_mask.nii.gz — Binary brain mask (skull-stripping result)
- sub-01_desc-aparcaseg_dseg.nii.gz — Anatomical parcellation and segmentation
- sub-01_desc-aseg_dseg.nii.gz — FreeSurfer subcortical segmentation
- sub-01_from-T1w_to-MNI152NLin2009cAsym_mode-image_xfm.h5 — Non-linear registration transform
- sub-01_from-MNI152NLin2009cAsym_to-T1w_mode-image_xfm.h5 — Inverse transform
- sub-01_from-T1w_to-fsnative_mode-image_xfm.txt — FreeSurfer native space transform

All outputs conform to the BIDS Derivatives specification, ensuring compatibility with downstream analysis tools such as Nilearn, FSL, and SPM.

CHAPTER 7

CONCLUSION AND FUTURE WORK

7.1 Summary of Phase 2 Work

Phase 2 of the Agentic fMRI Preprocessing project successfully delivered a working multi-agent system that autonomously manages the full fMRIPrep preprocessing lifecycle. The core contributions of this phase are:

- A LangGraph-based stateful orchestration framework with bounded retry logic and full execution provenance.
- Five specialized agents (BIDS, Resource, Pipeline, Diagnostic, Vision) with well-defined responsibilities and interfaces.
- A RAG-augmented command generation system that consults official fMRIPrep and BIDS documentation.
- A heuristic diagnostic engine covering seven common fMRI pipeline failure categories.
- A Toy Brain data engineering strategy that enables development and testing on consumer hardware.
- Demonstrated end-to-end autonomous processing with zero manual interventions, recovering from three critical runtime failures.
- A validated quality score of 0.94/1.0 from the Vision Agent QA module.

These results validate the core design hypothesis: that fMRI preprocessing can be successfully transformed from a static automation script into an intelligent, self-healing workflow through the application of multi-agent AI principles.

7.2 Future Work

Several directions are planned for Phase 3 and beyond:

7.2.1 Reinforcement Learning Policy Training

The Phase 2 infrastructure provides the foundation for training a proper RL agent. Future work will collect preprocessing decisions and quality outcomes across multiple subjects and datasets to build a training corpus. The RL agent will learn to select preprocessing parameters (number of CompCor components, motion censoring thresholds, smoothing kernel size) by maximizing a composite

reward signal. Proximal Policy Optimization (PPO) is the candidate algorithm given the continuous action space.

7.2.2 Full fMRI Functional Processing

Phase 2 demonstrated the system on the anatomical (--anat-only) workflow. Phase 3 will extend to the full functional preprocessing pipeline, including BOLD image alignment, ICA-based denoising, and first-level GLM modeling. This will require higher-memory compute resources or cloud deployment.

7.2.3 Cloud and HPC Deployment

The Docker-based architecture of the Agentic system is designed for portability to cloud and HPC environments. Future work will package the system for deployment on AWS, Google Cloud, or university HPC clusters, enabling processing of full-resolution datasets at scale.

7.2.4 Expanded Vision Agent

The current Vision Agent performs pass/fail scoring. Future work will train a deep learning classifier on labeled QC images from MRIQC to produce per-artifact severity scores, enabling the system to diagnose subtle quality issues such as susceptibility distortion, Gibbs ringing, and EPI signal dropout.

CHAPTER 8

REFERENCES / BIBLIOGRAPHY

- [1] O. Esteban, C. J. Markiewicz, R. W. Blair et al., "fMRIPrep: a robust preprocessing pipeline for functional MRI," *Nature Methods*, vol. 16, no. 1, pp. 111–116, Jan. 2019.
- [2] O. Esteban, R. Ciric, K. Finc et al., "Analysis of task-based functional MRI data preprocessed with fMRIPrep," *Nature Protocols*, vol. 15, no. 7, pp. 2186–2202, Jul. 2020.
- [3] A. I. Luppi, M. J. Albers et al., "Systematic evaluation of fMRI data-processing pipelines for consistent functional connectomics," *Nature Communications*, vol. 15, May 2024.
- [4] R. H. R. Deckers, P. van Gelderen et al., "An adaptive filter for suppression of cardiac and respiratory noise in MRI time series data," *NeuroImage*, vol. 33, no. 4, pp. 1136–1146, Nov. 2006.
- [5] H.-T. Wang, S. L. Meisler et al., "Continuous evaluation of denoising strategies in resting-state fMRI connectivity using fMRIPrep and Nilearn," *PLOS Comput. Biol.*, vol. 20, no. 3, p. e1011942, Mar. 2024.
- [6] Z. Yang, "A deep neural network for adaptive spatial smoothing of task-fMRI data," *Front. Neuroimaging*, vol. 4, Jan. 2025.
- [7] N. W. Churchill, D. Spring, A. Grady et al., "An Automated, Adaptive Framework for Optimizing Preprocessing Pipelines in Task-Based Functional MRI," *PLOS ONE*, vol. 10, no. 7, p. e0131520, Jul. 2015.
- [8] R. Botvinik-Nezer, F. Holzmeister et al., "Variability in the analysis of a single neuroimaging dataset by many teams," *Nature*, vol. 582, pp. 84–88, Jun. 2020.
- [9] L. Pineda, S. Basu, A. Romero, R. Calandra, and M. Drozdal, "Active MR k-space sampling with reinforcement learning," in *Proc. MICCAI*, 2020, pp. 23–33.
- [10] F. Knoll et al., "fastMRI: A publicly available raw k-space and DICOM dataset of knee images for accelerated MR image reconstruction using machine learning," *Radiol. Artif. Intell.*, vol. 2, no. 1, Jan. 2020.
- [11] J. Ren, A. N., C. Lin et al., "DeepPrep: an accelerated, scalable and robust pipeline for neuroimaging preprocessing empowered by deep learning," *Nature Methods*, Jan. 2025.
- [12] D. Van Essen, S. M. Smith et al., "The WU-Minn Human Connectome Project: an overview," *NeuroImage*, vol. 80, pp. 62–79, Oct. 2013.
- [13] J. Miller et al., "Multimodal population imaging in the UK Biobank," *Nature Neuroscience*, vol. 19, no. 11, pp. 1523–1536, Nov. 2016.

- [14] K. Murphy, T. Birn, and P. Bandettini, "Resting-state fMRI confounds and their mitigation," *NeuroImage*, vol. 80, pp. 345–359, Oct. 2013.
- [15] J. D. Power, K. Barnes, A. Snyder, B. Schlaggar, and S. Petersen, "Spurious correlations in functional connectivity networks arise from subject motion," *NeuroImage*, vol. 59, no. 3, pp. 2142–2154, Feb. 2012.
- [16] K. M. Gorgolewski et al., "Nipype: a flexible, lightweight and extensible framework for neuroimaging pipelines," *Front. Neuroinf.*, vol. 5, p. 13, 2011.
- [17] Y. Y. Yeh and C. T. Yang, "Object Memory and Change Detection," *Acta Psychol. (Amst)*, vol. 127, no. 2, pp. 420–440, Feb. 2008.
- [18] J. Carp, "The secret lives of experiments: Methods reporting in the fMRI literature," *NeuroImage*, vol. 63, no. 1, pp. 289–300, Oct. 2012.
- [19] S. Chen, B. Hu, and Y. Zhang, "A survey on deep learning for neuroimaging-based brain disorder analysis," *Front. Neurosci.*, vol. 14, p. 779, 2020.
- [20] G. H. Glover, "Overview of functional magnetic resonance imaging," *Neurosurg. Clin. N. Am.*, vol. 22, no. 2, pp. 133–139, Apr. 2011.
- [21] R. A. Poldrack et al., "Scanning the horizon: towards transparent and reproducible neuroimaging research," *Nat. Rev. Neurosci.*, vol. 18, no. 2, pp. 115–126, Feb. 2017.
- [22] P. Pruim et al., "ICA-AROMA: a robust ICA-based strategy for removing motion artifacts from fMRI data," *NeuroImage*, vol. 112, pp. 267–277, May 2015.
- [23] M. Akçakaya et al., "Scan-specific robust artificial-neural-networks for k-space interpolation (RAKI)," *Magn. Reson. Med.*, vol. 81, no. 1, pp. 43–53, Jan. 2019.
- [24] M. Mardani et al., "Deep generative models for compressed sensing MRI," *IEEE Trans. Med. Imag.*, vol. 38, no. 1, pp. 167–179, Jan. 2019.
- [25] B. Zhu et al., "Image reconstruction by domain-transform manifold learning," *Nature*, vol. 555, pp. 487–492, Mar. 2018.
- [26] D. Piccini et al., "Deep learning to automate reference-free image quality evaluation in MRI," *Radiol. Artif. Intell.*, vol. 2, no. 6, Nov. 2020.